

The setup: a simple counter

The tutor creates a social media `Post` struct with a `views` field, and attaches a method to increment it:

```
type post struct {  
    views int  
}  
  
func (p *post) inc() {  
    p.views += 1  
}
```

In `main`, he creates a post and calls `inc()` in a loop 100 times. Result: exactly 100, every single run. No surprise — the loop is **synchronous**. Iteration 2 cannot start until iteration 1 fully finishes. One worker, one ledger, no chaos.

The break: adding `go`

Then he does one tiny thing — adds the `go` keyword so each increment runs as a goroutine (plus a `sync.WaitGroup` so `main` doesn't exit before the goroutines finish — `wg.Add(1)` before launching, `defer wg.Done()` inside `inc`, `wg.Wait()` at the end).

Now the output is: 100... then 96... then 94... then 100... then 99. **Randomly wrong**. Same code, same loop, different answers. That's the race condition.

Why it breaks: `p.views += 1` is not one operation

This is the core insight of the whole video. That single line of Go compiles down to **three machine-level steps**:

1. **Read** the current value of `views` from memory into a CPU register
2. **Add** 1 to it in the register
3. **Write** the result back to memory

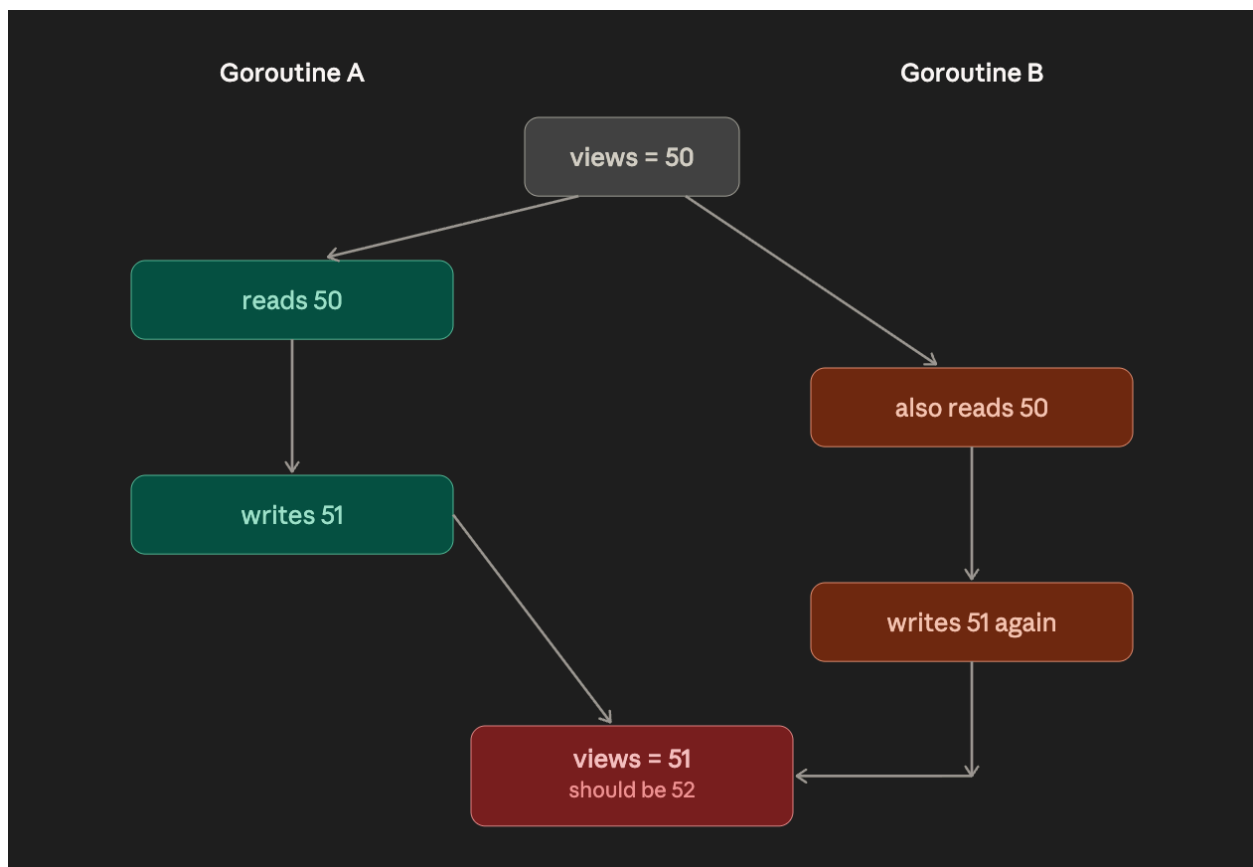
The Go scheduler can pause a goroutine *between any of these steps* and let another goroutine run. So picture this with `views = 50`:

- Goroutine A reads `views` → gets 50
- Scheduler switches. Goroutine B reads `views` → also gets 50
- A computes 51, writes back → `views = 51`
- B computes 51 (from its stale read), writes back → `views = 51`

Two increments happened, but the counter moved by one. B **overwrote** A's update — this is called a *lost update*. Do this across 100 goroutines and you randomly lose a few increments each run, which is exactly why he saw 94, 96, 99.

Analogy check, since you like to verify these: imagine a shared Google Doc, but instead of live sync, two people each download a copy, edit offline, and re-upload. Whoever uploads last silently wipes the other's edit. The "download" is the read, the "upload" is the write, and the gap in between is where the race lives.

Here's the interleaving visually:



The fix: `sync.Mutex` (mutual exclusion)

A mutex is a lock that only one goroutine can hold at a time. The convention the tutor follows — and it's a good one — is to put the mutex **inside the struct it protects**, right next to the data:

```
type post struct {
    views int
    mu     sync.Mutex
}

func (p *post) inc(wg *sync.WaitGroup) {
    defer wg.Done()
    p.mu.Lock()
    defer p.mu.Unlock()
    p.views += 1
}
```

How it behaves: when goroutine A calls `Lock()`, it acquires the lock and proceeds. If goroutine B calls `Lock()` while A holds it, **B blocks** — the Go runtime parks that goroutine until A calls `Unlock()`. So the read-add-write sequence becomes effectively atomic from the outside. Nobody can sneak in between the read and the write anymore. Run the program now and you get 100 every single time.

The classic analogy is the single key to a washroom: whoever holds the key is inside, everyone else queues at the door. The key is the mutex, the washroom is the critical section, and `Unlock()` is hanging the key back.

Two best practices the tutor emphasizes

1. `defer p.mu.Unlock()` instead of unlocking manually at the end. Why? If there's any code between `Lock()` and `Unlock()` that panics or returns early, a manual unlock never

executes — and the lock is held *forever*. Every other goroutine waiting on it blocks permanently. That's a deadlock-by-leak. `defer` guarantees the unlock runs no matter how the function exits, even via panic. Same logic as `defer wg.Done()`.

2. Lock the smallest possible section — only the lines that touch shared state. This is the more subtle, more important point. The mutex is a deliberate bottleneck: inside the locked region, your beautifully concurrent program becomes single-file. If your function does a DB query, builds a payload, calls an API, *and* updates a shared counter — and you wrap the entire function in the lock — your 50 goroutines are now effectively running sequentially, and you've thrown away the whole point of concurrency. Lock right before the shared write, unlock right after. Keep slow stuff (I/O, network calls) outside the lock.

Where this maps to your pipeline

You already live with this exact problem. In your bulk email worker pool, imagine you keep an in-memory counter like `sentCount` or a shared `map[string]int` tracking per-campaign delivery stats, and 50 workers increment it as they send. Without a mutex, you'd silently lose counts — exactly the 94/96/99 behavior, except in production it shows up as "dashboard says 9,847 sent but SES says 10,000" and nobody knows why. (Maps in Go are even worse — concurrent map writes don't just lose data, they *panic* the program.)

Also note: your semaphore-based concurrency limit and a mutex solve different problems. The semaphore limits *how many* goroutines run at once; the mutex ensures *only one at a time* touches a specific piece of shared state. You can have 5 workers running concurrently (semaphore) where each briefly takes the mutex only when updating the shared counter.

One last practical tip the video didn't cover: Go has a built-in race detector. Run `go run -race mutex.go` on the buggy version and it'll print exactly which goroutines raced on which variable, with stack traces. Worth running on your notification service tests at least once — it catches races that "work fine" 99% of the time.

Quick check of your understanding: in the tutor's final code, why does `defer wg.Done()` come *before* `Lock()` in source order but still work correctly? Think about when defers actually execute — tell me your reasoning and I'll confirm.